OnTrek: An app that simplifies your hikes

Alessio Pannozzo
Sapienza University of Rome
Italy
pannozzo.1960374@studenti.uniroma1.it

Antonio Pietro Romito
Sapienza University of Rome
Italy
romito.1932500@studenti.uniroma1.it

Federico Pizzari
Sapienza University of Rome
Italy
pizzari.1936451@studenti.uniroma1.it

Gioele Maria Zoccoli
Sapienza University of Rome
Italy
zoccoli.1850491@studenti.uniroma1.it

Abstract

Trekking and outdoor activities present unique challenges related to navigation, safety, and group coordination. Existing digital solutions often fall short in providing reliable, real-time assistance in remote or high-difficulty environments. This work introduces OnTrek, a smartwatch–smartphone system designed to simplify hiking while enhancing user safety. The smartwatch client provides intuitive, clutter-free directional guidance and group-awareness features, while the smartphone application manages trail planning, group coordination, and profile management. A key component of OnTrek is an integrated fall detection module, built upon a hybrid deep learning architecture combining CNN, GRU, and LSTM branches, optimized through sensor fusion, Kalman filtering, and efficient preprocessing of accelerometer and gyroscope data. Evaluation on the FallAllD dataset demonstrates competitive performance, achieving over 97% accuracy while remaining lightweight enough for mobile deployment. Compared with more complex baselines, the optimized Coarse+Gyro model balances robustness, inference speed, and energy efficiency, enabling real-time operation on wearable devices. Overall, OnTrek demonstrates the feasibility of combining wearable sensing, mobile computing, and intelligent fall detection into a unified platform that supports safe, collaborative, and user-friendly hiking experiences.

1 Introduction

1.1 Problem Context

Trekking and outdoor activities are experiencing growing popularity, attracting both occasional hikers and experienced mountaineers. However, despite the widespread availability of digital navigation tools, hikers often face critical issues such as getting off trail, difficulties in retracing their path, or encountering impassable routes. The collected survey responses highlight that many participants have at least once lost track of the marked path and required strategies ranging from backtracking to the use of smartphone applications or even asking for external help. These situations not only cause delays but can also escalate into safety risks, especially in remote or high-difficulty environments.

Falls are the primary cause of mountain-related injuries and deaths in Italy. According to the National Alpine and Speleological Rescue Corps (CNSAS), in 2024, 44.3% of all rescue missions were related to hiking accidents, with falls being the most common cause [1]. This statistic underscores the critical need for preventive measures and safety awareness among hikers.

Planning and orientation practices also vary widely. While some hikers rely on traditional resources such as paper maps or guidance from companions, others use smartphone apps or wearable devices. Nevertheless, the reported experiences reveal gaps in reliability, accessibility, and real-time support of current solutions. For instance, not all users carry or trust smartwatches, and map-based planning may fail to provide timely information about trail conditions or emergencies.

The survey further indicates a strong interest in digital features such as directional guidance, deviation alerts, and automated emergency assistance. These needs reflect a broader challenge: designing technological systems that are both user-friendly and robust under the constraints of outdoor environments, where connectivity, battery autonomy, and usability can be critical factors.

Understanding hikers' behaviors, challenges, and expectations is therefore essential to inform the development of next-generation trekking support systems. Addressing these gaps not only improves user experience but also contributes to safety, risk prevention, and sustainable enjoyment of natural environments.

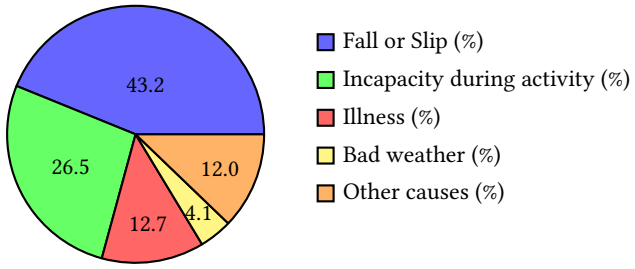

Figure 1: Distribution of the main causes of interventions [1].

2 OnTrek

OnTrek is a smartwatch application designed to assist individuals who require guidance during trekking or hiking activities. The system includes a companion smartphone application that allows users to create sessions with friends and select the trail to follow during the activity.

The architecture of OnTrek is based on a client-server model. The smartwatch client handles real-time navigation, displaying a directional arrow that indicates the path to follow. The arrow changes color from green to red depending on the user's distance from the trail, providing an intuitive visual cue that simplifies navigation.

Unlike traditional map-based applications, the smartwatch interface avoids clutter and focuses on immediate directional guidance, enhancing usability in outdoor environments.

The smartphone companion app manages session creation, trail selection, and coordination among participants. It communicates with the smartwatch client to synchronize the user's position and provides a radar view of all group members in real-time. This radar highlights potential issues, including help requests and signal loss, enabling timely interventions during group activities.

OnTrek relies on cloud-based services for data synchronization and location updates, ensuring that all members of a session have access to consistent and accurate information. The system is designed to handle intermittent connectivity, allowing for robust operation even in remote areas. For further details and the implementation of the system, the source code is publicly available on GitHub.

2.1 Application Architecture

The application is built following a client-server architecture, consisting of an Android frontend and a Go-based backend. The frontend, developed in Kotlin, is responsible for the user interface and presentation logic, allowing users to interact with the application's features in an intuitive way. It communicates with the backend through HTTP requests to RESTful APIs, exchanging data in JSON format.

The backend handles the business logic and data access. It receives requests from the frontend, processes them, interacts with the database, and returns the necessary responses. This centralization ensures both data consistency and security.

The communication flow between the frontend and backend is straightforward: when a user performs an action on the Android app, the frontend sends a request to the backend; the backend processes the request and returns a response, which the frontend uses to update the user interface. This design enforces a clear separation between presentation and business logic, facilitating maintenance and scalability.

The application leverages several technologies to implement this architecture. On the frontend side, Kotlin and the Android SDK form the core of the app, while libraries such as Retrofit simplify HTTP communication with the backend. On the server side, Go, together with the Gin framework and the GORM ORM, efficiently handle requests and database operations. The entire system relies on RESTful APIs, using JSON as the standard data exchange format, ensuring interoperability and readability.

Smartphone and smartwatch apps communicate mainly through the Google Wear OS platform, using Google Play Services APIs such as DataClient and MessageClient. These APIs rely on Bluetooth Low Energy (BLE) for efficient and secure information transfer between devices. Data such as fall detection results or track start commands are serialized and sent through dedicated channels, ensuring real-time synchronization between the installed apps.

2.2 Mobile

The mobile application is designed with a simple and lightweight architecture comprising four main interfaces. The first interface allows users to navigate through the groups they belong to and schedule new hikes. The second interface provides functionality for uploading and managing user-generated tracks(see figure 2). Once a GPX file is uploaded, the server processes it by generating a route visualization using the Mapbox [5] API and computing all relevant statistics associated with the track. Here, we can also start the navigation automatically, by launching the smartwatch app with the corresponding track. The third interface enables users to search for and connect with new friends, while the fourth interface provides access to the user profile, smartwatch pairing functionalities, and visualization of the friend network.

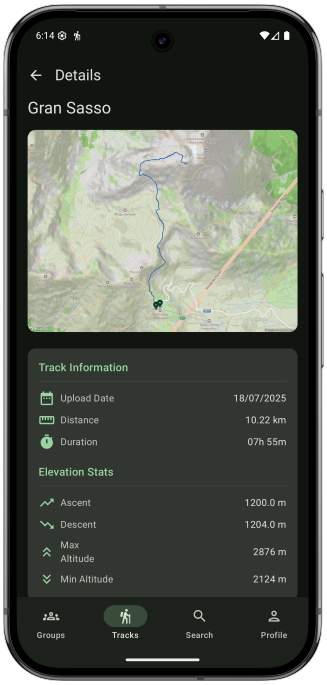

Figure 2: Smartphone interface.

During hiking, the mobile application also collects data from the gyroscope and accelerometer to feed the fall detection model (which is detailed in the following), also hosted on the same app.

2.3 Wear

The smartwatch application, as illustrated in Figure 3, is designed with a simple and highly user-friendly interface, showing only four main components, of which two are interactive. Along the screen border, a progress bar visualizes the distance traveled in the current hike. At the bottom, an emergency SOS button is available; if pressed for five seconds, it sends a help request to the server, which is subsequently broadcast to other users. In the center of the screen, a radar component provides a real-time visualization of the session members and their distance from the user. Each member can be represented by four clearly distinguishable states: normal, SOS, disconnected, and assisting another participant. An arrow is always displayed as the primary component of the interface, indicating the right direction to follow.

Navigation is designed to guide users along a predefined path using GPS and compass data. It works by continuously comparing the user's current location to a series of track points parsed from

Figure 3: Smartwatch interface.

a GPX file. The algorithm identifies the nearest track point and determines the next point the user should move towards. If the user is very close to a point (within a defined threshold of 15 meters), the function advances to the next point until the user is far enough away. For intermediate cases, it calculates distances to neighboring points and uses geometric offsets to decide whether the user should be guided to the current point or the next one, ensuring smooth and accurate navigation along the track. The logic adapts to both normal progression and edge cases, like being at the start or end of the route. If the user deviates from the track beyond a certain distance, the algorithm triggers notifications (Figure 4) and user can snooze them to avoid excessive interruptions. In addition, progress along the track is calculated based on the distance traveled relative to the total track length.

When a user issues an SOS request, a dialog is presented on the screen of other hike members, allowing them to decide whether to provide assistance. Accepting the dialog will make the interface give the direction to the user who requested help. If the dialog is accidentally dismissed, it can be reopened by selecting the icon corresponding to the requesting user. Accepting the dialog will start the dynamic rerouting by recalculating the path to redirect to the user that requested help.

Conversely, when the user themselves requests help, the interface changes: the radar highlights only those participants moving toward the distressed user. This design ensures that the requester receives immediate visual feedback that their call for help has been acknowledged.

The application also supports solo sessions. In such cases, the radar and the SOS button are not rendered, as they are not required in this context, leaving only the minimal set of information relevant to individual activity tracking.

2.4 Multimodal Interactions

The OnTrek apps leverages a multimodal approach to enhance user experience and safety through various interaction methods:

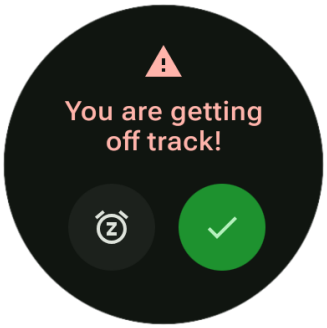

Figure 4: Smartwatch interface.

- Fall Detection: using mobile accelerometer and gyroscope sensors, the app continuously monitors user movement to detect falls.
- Movement Tracking: the app responds to changes in user position and orientation (walking, turning) via GPS, magnetometer and gyroscope sensors, updating navigation and visual cues in real time.
- Haptic Feedback: the app uses vibration to notify important events such as going off track, receiving help requests, or fall detection.
- Touch Interaction: users can interact with the interface by tapping, for example to send SOS requests or confirm alerts.
- Gesture: the smartwatch app, in order to save battery, automatically dims the screen brightness and deactivates the magnetometer and gyroscope sensors when it detects that the user lowers their arm. Then, it automatically reactivates them when it is lifted.

These multimodal interactions allow the app to dynamically adapt to user actions and physical state.

3 Fall Detection

3.1 Related Work

Conventional fall detection (FD) approaches have largely relied on traditional machine learning algorithms such as SVM, Naïve Bayes, kNN, and decision trees. While these models demonstrated the feasibility of accelerometer-based FD, they depend on handcrafted features and domain expertise, which often limits scalability in real-world scenarios.

Deep learning methods have addressed these limitations by enabling automatic feature extraction from raw sensor data. CNNs are widely used to capture spatial features of accelerometer signals, achieving high performance in differentiating falls from activities of daily living (ADLs). On the other hand, RNNs, particularly LSTM and GRU architectures, are well suited for temporal modeling. GRU-based models in particular have shown performance comparable to LSTM while requiring fewer parameters and lower computational cost.

Hybrid CNN–RNN approaches have been proposed to exploit both spatial and temporal features. For example, Xu et al. [4] introduced CNN-LSTM for FD, showing improved accuracy compared to

standalone CNNs or RNNs. However, sequential designs like CNN-LSTM may diminish the contribution of CNN-extracted features, as the temporal modeling dominates the representation. To address this, Avilés-Cruz et al. proposed a parallel coarse-fine CNN for human activity recognition, where multiple convolutional branches preserve spatial information at different granularities.

Our work builds upon this line of research. Drawing inspiration from the coarse-fine CNN structure, we adopt the principle of parallel feature extraction and extend it with the integration of CNN branches and GRU layers. This design aims to preserve multi-scale spatial representations while also capturing temporal dependencies, addressing the limitations of sequential CNN–RNN pipelines and enhancing the robustness of fall detection.

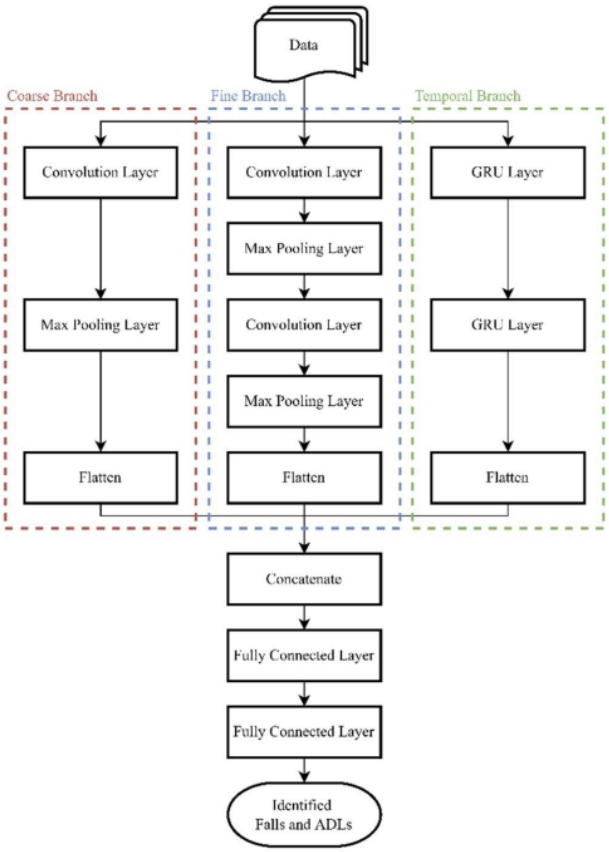

Figure 5: Model proposed in the paper [4]

3.2 Proposed model

While our work was inspired by the ensemble model proposed before, we introduce several key modifications to enhance its ability to capture motion dynamics from wearable sensors.

First, instead of limiting the input to tri-axial accelerometer data, we extend the model to also include the three axes of gyroscope signals. This provides a more comprehensive description of human motion by combining information on both linear acceleration and angular velocity, which is especially useful for distinguishing subtle daily activities from fall events.

Second, in addition to the original coarse-grained CNN, fine-grained CNN, and GRU branches, we introduce an additional recurrent branch based on Long Short-Term Memory (LSTM) units. This branch is designed with two stacked layers and a hidden dimension of 64, processing sequences in batch-first mode. Unlike the GRU branch, which offers efficiency in modeling short-term dependencies, the LSTM branch is particularly suited to capture longer-term temporal dynamics in the sensor data.

Finally, the outputs of the CNN, GRU, and LSTM branches are concatenated to form a unified feature representation before classification. This design allows the model to integrate spatial features at multiple granularities with temporal dependencies extracted through complementary recurrent architectures, ultimately improving the robustness of fall detection.

3.3 FallAllD Dataset

The FallAllD dataset [6] was designed to overcome the limitations of earlier fall detection corpora by providing a comprehensive and realistic collection of human falls and activities of daily living (ADLs). It consists of 26,420 files, each containing 20 seconds of motion signals recorded during simulated falls and ADLs. Data were acquired from 15 participants (8 males and 7 females), with ages ranging from 21 to 53 years. The protocol systematically covered a wide variety of scenarios, including 35 types of falls and 44 types of ADLs, ensuring high representativeness and diversity. Importantly, both falls with and without recovery were included to reflect realistic conditions.

Each participant wore up to three wearable devices (neck, waist, and wrist), though the dataset is intended for single-device fall detection systems. Each data-logger is equipped with four sensors:

- A tri-axial accelerometer, sampled at 238 Hz, with a measurement range of $\pm 8g$;
- A tri-axial gyroscope, sampled at 238 Hz, with a measurement range of $\pm 2000°/s$;
- A tri-axial magnetometer, sampled at 80 Hz, with a measurement range of ±4 Gauss;
- A barometer, sampled at 10 Hz, measuring air pressure and temperature.

Overall, FallAllD represents one of the most complete and diverse datasets for fall detection research, enabling the evaluation of both classical and deep learning approaches under realistic conditions.

3.4 Kalman Filtering

Sensor signals, such as those collected by accelerometers and gyroscopes, are often subject to various sources of noise, including vibrations, temperature variations, and environmental interference. This noise can significantly affect the quality of the data and, consequently, reduce the accuracy of fall detection models. To address this issue, many studies in the literature have applied Kalman filtering as a preprocessing step [3]. The Kalman filter is a recursive algorithm that estimates the true state of a system by combining observed measurements with predictions from a system model, effectively smoothing the signal and reducing the influence of noise [2].

Formally, the filter alternates between a *prediction* step and an *update* step. Given the previous state estimate $\hat{x}_{k-1|k-1}$, the filter

predicts the new state as

$$\hat{x}_{k|k-1} = A\hat{x}_{k-1|k-1} + Bu_k,$$

with associated error covariance

$$P_{k|k-1} = AP_{k-1|k-1}A^T + Q,$$

where A is the state transition matrix, B the control-input model, and Q the process noise covariance.

When a new measurement z_k becomes available, the update step refines the estimate using the Kalman gain K_k:

$$K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1},$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1}),$$

$$P_{k|k} = (I - K_kH)P_{k|k-1},$$

where H is the observation model and R the measurement noise covariance. This recursive scheme ensures that the estimated state $\hat{x}_{k|k}$ converges toward the true underlying signal, even in the presence of noise.

In the context of fall detection, Kalman filtering is particularly useful for obtaining cleaner acceleration and angular velocity signals, which makes feature extraction and classification more reliable. This preprocessing step ensures that the subsequent segmentation and modeling stages operate on denoised signals, improving the robustness of our pipeline and reducing the likelihood of false classifications.

3.5 Data Preprocessing

In order to prepare the FallAllD dataset for training and evaluation, several preprocessing steps were applied. First, only the relevant sensors were retained: accelerometer and gyroscope signals were selected, while the unused sensors provided in the dataset (magnetometer and barometer) were discarded.

To ensure compatibility with mobile deployment, accelerometer and gyroscope measurements were converted into the same format as those natively provided by Android sensors. This choice avoids additional conversion steps on the mobile device before the model forward pass, reducing processing overhead.

Since power efficiency is a critical factor for wearable applications, the original sampling frequency of 238 Hz was downsampled to 20 Hz. This configuration allows the model to operate with lower energy consumption while preserving sufficient temporal resolution to capture fall dynamics.

Before applying segmentation, a Kalman filter was employed on the preprocessed signals to reduce measurement noise and improve the robustness of feature extraction.

Finally, a sliding window segmentation was applied, with a window length of 7 s and an overlap of 50%. This segmentation resulted in a total of 6,260 windows, including 5,328 corresponding to activities of daily living (ADLs) and 932 to fall events. This balanced design provides the network with temporally aligned input sequences for supervised training.

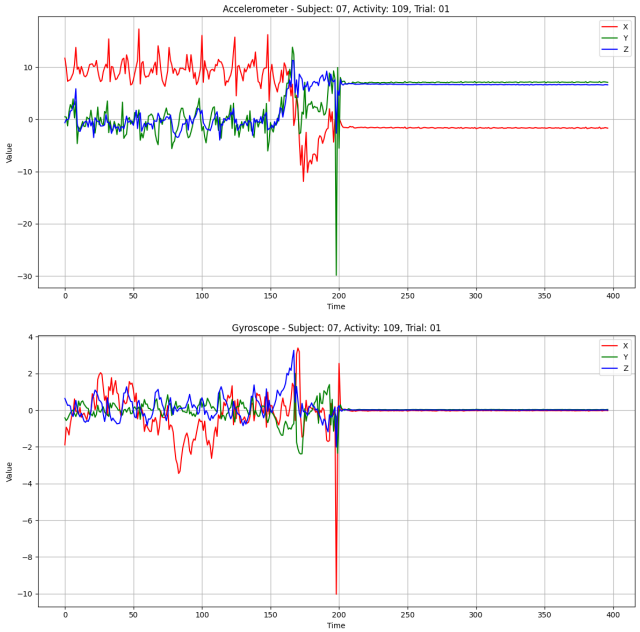

Figure 6: Original data

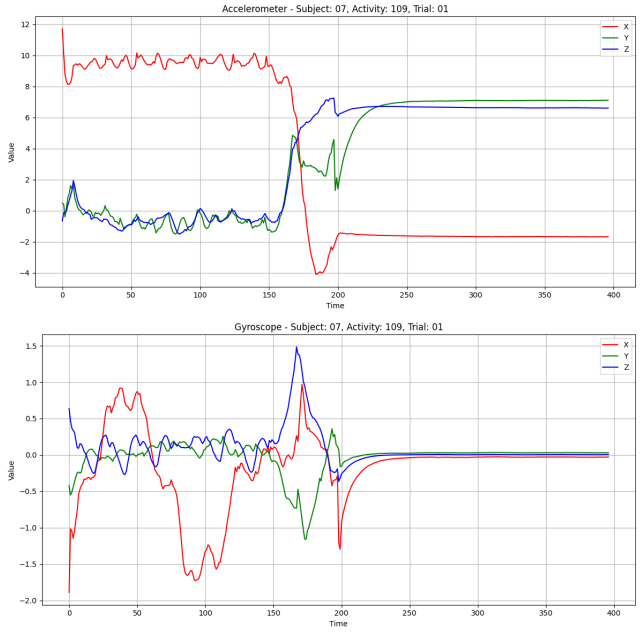

Figure 7: Processed data

3.6 Dataset Organization

An important characteristic of the dataset is its pronounced class imbalance, as shown in Figure 8. Normal daily activities are substantially more frequent than fall events, which introduces a bias that can hinder the learning process. To address this issue, we adopted a weighted loss function that assigns greater importance to

fall samples, thereby compensating for their under-representation and ensuring a more balanced contribution of both classes during training.

Regarding the dataset organization, the samples were divided into three stratified subsets in order to preserve the original class distribution, as illustrated in Figure 9. Specifically, 70% of the samples were assigned to the training set, while the remaining 30% was further split into validation and test subsets, corresponding to approximately 12% and 18% of the entire dataset, respectively. Data were fed into the model using PyTorch *DataLoader* objects with a batch size of 64. Shuffling was applied only during training, whereas validation and test sets were processed sequentially.

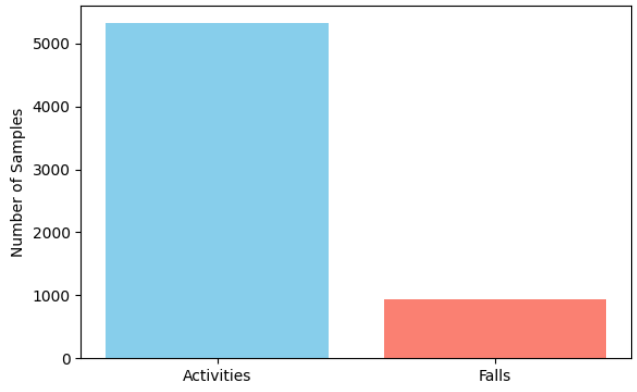

Figure 8: Class imbalance

Figure 9: Class distribution over subsets

4 Performance Evaluation

To assess the effectiveness of the proposed model, the *confusion matrix* was used as the basis for deriving the main classification metrics. It is defined in terms of:

- True Positives (TP)
- True Negatives (TN)
- False Positives (FP)
- False Negatives (FN)

4.1 Confusion Matrix

The confusion matrix for a binary classification task can be represented as:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

From this matrix, the following evaluation metrics are computed:

4.2 Evaluation Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

4.3 Remarks

- **Accuracy**(1) expresses the overall proportion of correctly classified samples with respect to the entire dataset.
- **Precision**(2) instead focuses on the reliability of positive predictions, quantifying the fraction of correctly identified positives among all samples predicted as positive.
- **Recall**(3), also known as sensitivity, measures the ability of the model to detect all actual positive instances, thus reflecting its capacity to minimize missed detections.
- **F1-score**(4) combines precision and recall into a single measure by computing their harmonic mean, making it particularly valuable when dealing with imbalanced datasets where accuracy alone might provide a misleading assessment.

4.4 Models performance

In this section, we present the performance of our chosen model. For comparison purposes, we also evaluated several alternative models to assess how our model performs relative to other approaches.

4.4.1 CBAM-IAM-CNN-BiLSTM Model. In addition to our proposed approach, we also evaluated the CBAM-IAM-CNN-BiLSTM model introduced by Li et al. [3]. This architecture was specifically designed to address two fundamental challenges in fall detection: the need to jointly capture spatial and temporal dependencies from inertial signals, and the difficulty of distinguishing between highly similar daily activities and actual fall events. To this end, the model leverages CNN layers to extract discriminative spatial features, while bidirectional LSTMs are employed to capture sequential dependencies across time. An improved convolutional block attention module (CBAM-IAM) is integrated to enhance the feature learning process by selectively weighting the most informative channels and spatial patterns, while reducing redundant computations through the use of one-dimensional convolution. By combining these components, the model is able to fuse low-level and high-level representations into a unified feature space before final classification

via softmax, resulting in a network that is both expressive and computationally efficient for wearable fall detection tasks.

4.4.2 CNN-BiLSTM Model. We designed a hybrid architecture that combines Convolutional Neural Networks (CNNs) and bidirectional Long Short-Term Memory (BiLSTM) networks to capture both local and temporal patterns from multivariate time-series sensor data. The model consists of two parallel branches that process accelerometer and gyroscope signals separately. The accelerometer branch employs a series of 1D convolutional layers followed by ReLU activations and an adaptive average pooling layer to extract high-level local features. Conversely, the gyroscope branch is modeled with a two-layer bidirectional LSTM, which captures temporal dependencies in the gyroscope measurements. The outputs from both branches are flattened and concatenated to form a comprehensive feature representation. This combined feature vector is passed through a fully connected layer with ReLU activation and dropout for regularization, followed by a final linear layer producing the class logits.

4.4.3 BiLSTM model. As a baseline, we implemented a simple LSTM model that directly models the temporal dependencies of the multivariate sensor data. The architecture consists of a two-layer bidirectional LSTM with 128 hidden units per direction, which processes the full input sequence of six channels (accelerometer and gyroscope signals). The LSTM output is flattened and passed through a fully connected layer with ReLU activation and dropout for regularization, followed by a final linear layer that produces the class logits. Similar to the hybrid model, the network is trained using a weighted cross-entropy loss to account for class imbalance. This simpler architecture serves as a comparative baseline to evaluate the benefits of combining CNN feature extraction with LSTM temporal modeling.

Model	Total parameters	Estimated size(MB)
BiLSTM	5.1 M	20.490
CNN-BiLSTM	6.3 M	25.091
CBAM-IAM	19.8 M	79.271
Coarse	691 K	2.765
Coarse+Gyro	746 K	2.986

Table 1: Models details

Model	Accuracy	Precision	Recall	F1-Score
BiLSTM	97,42%	97.63%	97.42%	97,48%
CNN-BiLSTM	96,98%	97,16%	96,98%	97,03%
CBAM-IAM	97,24%	97,49%	97,24%	97,31%
Coarse	97,07%	97,29%	97,07%	97,13%
Coarse+Gyro	97,16%	97,39%	97,16%	97,22%

Table 2: Models performances

The model complexity details in Table 1, combined with the performance results in Table 2, provide a clear view of the trade-offs between accuracy and computational efficiency. While the BiLSTM achieved the best overall performance (97.42% accuracy, 97.48% F1-score), it requires 5.1M parameters and approximately 20 MB of memory. The CNN-BiLSTM and CBAM-IAM models employ more complex architectures, with 6.3M (25 MB) and 19.8M (79 MB) parameters respectively, but their performance improvements are marginal. In particular, CBAM-IAM, despite being the largest model by an order of magnitude, did not significantly outperform BiLSTM, raising concerns about its efficiency for real-time or resource-constrained deployment.

In contrast, the coarse-grained models (Coarse and Coarse+Gyro) achieved competitive performance (97.07% – 97.22% accuracy, 97.13% – 97.22% F1-score) with dramatically fewer parameters—below 1M and under 3 MB in size. Notably, the addition of gyroscope data provided a slight but consistent performance gain, confirming the value of multi-sensor fusion while preserving efficiency.

Overall, these findings suggest that BiLSTM offers the best balance between performance and moderate complexity, while the Coarse+Gyro model stands out as the most resource-efficient option, making it particularly attractive for embedded or mobile fall detection systems. Meanwhile, models with very high parameter counts, such as CBAM-IAM, provide limited practical advantage given their heavy memory footprint.

5 System Implementation and Optimization

Initially, we considered running the model directly on the smartphone, collecting sensor data from the smartwatch and forwarding it to the phone for inference. However, testing this approach with real users proved to be inefficient for several reasons. First, unexpected arm movements by the user could be misinterpreted as falls by the model. Second, transmitting data from the smartwatch to the smartphone introduced latency, which could result in delayed detection and notifications. It is important to note that these initial tests were conducted using the heaviest model in our study, the CBAM-IAM.

To address these issues, we decided to adopt a lighter model, referred to as Coarse+Gyro. This choice not only reduced latency but also significantly lowered energy consumption and memory requirements, shrinking the model size from 79 MB to approximately 3 MB. Additionally, the inference time for the Coarse+Gyro model was considerably faster compared to CBAM-IAM, further improving the responsiveness of the system. In this second test, the sensor was positioned at waist level to provide more stable and reliable measurements. By implementing the entire pipeline directly on the smartphone, which is usually kept in a pocket, the device now sends notifications only when a fall is actually detected, ensuring timely and accurate alerts without unnecessary transmissions.

Figure 10: Fall detection alert interface.

5.1 Fall detection alert interface

Upon detection of a fall event, the system activates an alert interface, as illustrated in Figure 10. At the top of the display, a countdown timer (initially set to 30 seconds) is shown, during which the user is prompted to confirm their condition. The central message "Fall detected! Are you okay?" is prominently displayed to encourage immediate interaction. Two large and visually distinct response options are provided: a red thumbs-down icon indicating the need for assistance, and a green thumbs-up icon confirming that the user is safe.

If no response is received within the allotted 30-second window, the system automatically sends an help requests to other group members. This mechanism introduces an implicit interaction modality: inaction itself (i.e., the absence of user input) is interpreted as a potential emergency condition, triggering the alert procedure. Such an approach reduces the cognitive and physical burden on the user, ensuring safety even in scenarios where active interaction is not possible due to injury, shock, or reduced mobility.

6 Conclusion

This work presented OnTrek, an integrated solution for hiking safety and coordination that combines a smartphone application with a smartwatch client. The system addresses critical challenges faced by hikers, such as disorientation, lack of group awareness, and emergency management. Through a simplified smartwatch interface, users receive clear navigation cues and real-time updates on group members' status, while the Android application provides track management and session coordination. Additionally, the incorporation of fall detection strengthens the safety dimension, allowing timely responses in potentially critical scenarios. The evaluation of multiple deep learning models showed that OnTrek can achieve both high accuracy and efficient deployment on mobile devices, balancing robustness with energy constraints. Overall, OnTrek demonstrates the feasibility and usefulness of combining wearable and mobile technologies to support safe, collaborative, and enjoyable trekking experiences.

7 Future Improvements

Although OnTrek achieves promising results, several avenues remain open for improvement. Future work will focus on:

- **Extended dataset collection**: building a larger and more diverse dataset of real hiking activities and falls, including participants of different age groups and terrains, to improve generalization.
- **Enhanced connectivity management**: integrating opportunistic networking techniques or satellite-based communication for areas with limited or no cellular coverage.
- **Integration with external services**: enabling connections with official rescue systems and health-monitoring platforms to extend the impact of emergency alerts.
- **Multimodal sensing**: combining physiological data (e.g., heart rate, oxygen saturation) with motion signals to improve fall detection reliability and to provide broader health monitoring during outdoor activities.

References

[1] Corpo Nazionale Soccorso Alpino e Speleologico. 2025. Disponibili i dati 2024 delle attività del soccorso alpino e speleologico. *CNSAS News*. https://news.cnsas.it/disponibili-i-dati-2024-delle-attivita-del-soccorso-alpino-e-speleologico/.

[2] Rudolph E. Kalman. 1960. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82, 1, 35–45. doi:10.1115/1.3662552.

[3] Congcong Li, Minghao Liu, Xinsheng Yan, and Guifa Teng. 2022. Research on cnn-bilstm fall detection algorithm based on improved attention mechanism. *Applied Sciences*, 12, 19, (Sept. 2022), 9671. Publisher: MDPI. doi:10.3390/app12199671.

[4] Chien-Pin Liu, Ju-Hsuan Li, En-Ping Chu, Chia-Yeh Hsieh, Kai-Chun Liu, Chia-Tai Chan, and Yu Tsao. 2023. Deep learning-based fall detection algorithm using ensemble model of coarse-fine cnn and gru networks. *arXiv preprint arXiv:2304.06335*. https://arxiv.org/abs/2304.06335.

[5] Mapbox, Inc. 2025. Mapbox documentation. https://docs.mapbox.com/. Accessed: 2025-08-27. (2025).

[6] Majd Saleh, Manuel Abbas, and Régine Le Bouquin Jeannès. 2021. Fallalld: an open dataset of human falls and activities of daily living for classical and deep learning applications. *IEEE Sensors Journal*, 21, 2, 1849–1858. doi:10.1109/JSEN.2020.3018335.